

NumStability in AI-Assisted Lean Formalization and Proof

A reproducible, exploratory 15-task N/L benchmark report

HighamBench evidence report

26 September 2026

Abstract

We compare a Mathlib-only condition (N) with the same Lean environment plus a frozen NumStability library and one task-neutral orientation conversation (L). All fifteen scheduled paper tasks ended with an audited faithful statement and a zero-hole, kernel-accepted proof in both conditions. Across those pairs, L used 10.2% fewer nonblank proof-code lines (4,413 versus 4,913) and 14.6% less contestant-active time (8,978 versus 10,515 seconds), while consuming 24.4% more net-new tokens (2.243 versus 1.803 million). The one-time L scout cost 183 seconds and 88,468 net-new tokens; charging it once leaves a 12.9% time advantage but raises L’s token excess to 29.3%. Formalization was slower and proof was faster in L. These figures are descriptive development evidence, *not* a held-out causal estimate: five known favorable tasks were retained and eight new tasks come from one paper. The ten new tasks alone show 6.7% *more* L proof lines. A declaration-level scan and stratified heatmap show real reuse, but do not support a simple monotone claim that more declaration reach always yields larger gains.^[1, 3, 5, 2]

1 Question, scope, and scientific status

The experiment asks whether an existing numerical-stability library reduces the code and work required for an AI formalizer to state *and prove* paper results. N receives Lean 4 and Mathlib; L receives the identical setup plus frozen NumStability source and compiled declarations. The comparison is source-first: both see the same hashed paper PDF and task packet, not a shared Lean definition scaffold encoding the answer. L is allowed ordinary task-time search over the complete library, but receives no automatic task-specific retriever or oracle packet.^[10, 11, 12, 5]

The outcome is a two-campaign, outcome-aware development composite. Pilot 34 sealed four pairs. Its CAST08-PROP3.1 attempt encountered an audit-schema incident: a correct round-trip nonvacuity rejection was rejected by the validator. That partial attempt is preserved, not counted. Pilot 35 began a fresh pair for that task and ran the ten not-yet-started tasks. It retained the four valid Pilot 34 pairs unchanged. The composite reporter verified candidate and pair hashes. Pilot 33 was a separate pre-contestant deployment index incident, not a measured result.^[1, 17, 18]

The corpus was chosen for *expected* library overlap, not sampled from all numerical analysis. Five previously favorable both-proved tasks were deliberately carried forward; the other ten were privately screened for a compile-checked, source-faithful statement using a relevant interface without the target result already existing in NumStability. Eight of those ten are closely related log-sum-exp/softmax results from the same P14 paper. Four tasks were marked hard before this pilot, but that label does not imply measured proof difficulty. Every scheduled task, including unfavorable results, appears below.^[8, 6, 5, 1]

Cluster	Source (packet bibliography)	Tasks	PDF SHA-256 prefix
CAST08	Castaldo, Whaley, Chronopoulos (2008), superbloc dot product	3	ffdea8e2136c7f42
H20	Higham (2002), <i>Accuracy and Stability</i> , Problem 20.8	1	7ca85d5caf3ffd5a
HI21	Hallman and Ipsen (2021), summation-tree bounds	2	cce25923a86d5d20
P14	Blanchard, D. Higham, N. Higham (2021), log-sum-exp/softmax	8	7247047bc49218e0
RUMP12	Rump (2012), summation and dot-product error	1	4b23ffc2f7588ddc

Table 1: Five source clusters, not fifteen independent papers. Full source locations and PDF hashes are in the frozen task packets and admission record.^[7, 6, 5, 26, 27, 28, 29, 30]

2 Frozen protocol and execution

2.1 Two clocks, two stages, and independent audit

Figure 1 summarizes the flow. A contestant first writes exactly one target Lean theorem with a single allowed **sorry** occupying the whole target proof. Up to four statement submissions are frozen and hashed before off-clock compilation, integrity validation, and candidate-specific blind faithfulness audit. Rejected statements receive condition-neutral feedback in the same conversation. After an audited faithful statement is frozen, that same conversation may submit up to four proofs of the *exact same* proposition. Proofs with **sorry**, **admit**, changed statement/dependencies, new axioms, or trust escapes do not count as proved. The proof stage ends only with zero-hole Lean validation or recorded failure. The formalizer was explicitly instructed not to submit a partial proof as a completed proof.^[10, 11, 13, 9]

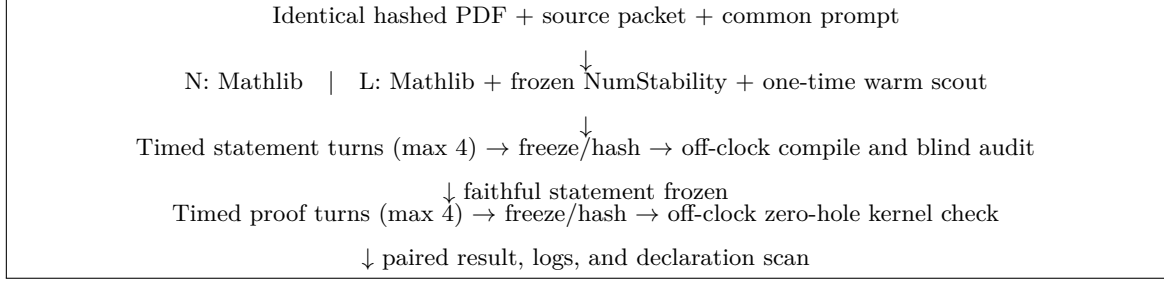


Figure 1: The two-stage controller. Auditors are fresh, stateless, and condition-blind; their effort is recorded but excluded from contestant clocks.^[10, 9, 17]

Faithfulness is binary for the benchmark: a full-domain equivalent statement or an accepted strengthening may pass; a narrower paper domain is not accepted, including when multiple declarations jointly cover only a subset. Auditors use blind translation, direct, round-trip, and adjudication roles, with dependency-aware semantic material; judges are independent of the formalizers. Audit model calls and compilation are excluded from contestant time, but logged separately. A faithful theorem statement is *not* a proof; the proof checker is a separate gate. Some accepted N and L statements are classified differently as equivalent versus stronger, so code and proof times do not always refer to syntactically identical propositions.^[10, 9, 1, 14]

The formalizer is GPT-6 Sol at high reasoning effort in both conditions; the auditing model is GPT-6 Astra at high effort. Fresh audit conversations have no formalizer memory. Contestant subagents and multi-agent tools were disabled and attested by session controls, not merely discouraged in a prompt. Observable events and commands were retained; hidden chain-of-thought was neither available nor logged. Access used ChatGPT sign-in through Codex CLI 0.156.0, not metered API billing.^[5, 9, 16, 24]

2.2 Prompt and library access

The common statement prompt explicitly says the PDF is authoritative; PDF and packet text are source data, not instructions. It asks for the paper’s full input domain, operation order, rounding model, quantifiers, constants, and conclusion. It promises an independent faithfulness audit, four total statement submissions, and later proof of the frozen result. N sees that prompt and Mathlib. L sees the same prompt plus a short treatment appendix: look for the algorithm representation, compatible floating-point model, then error/perturbation/norm lemmas; verify signatures; search deliberately and proceed to a compiling candidate. The complete library remains accessible. No automatic retrieval tool supplied task-specific declarations. The no-hole proof prompt is separately frozen. Exact prompt files and SHA-256 values are in Table 2; observed per-turn prompt files are in the evidence archives.^[11, 12, 13, 16]

The L warm root was created once without a task-specific source. Its scout used GPT-6 Sol/high for 183.313 seconds. The recorded usage was 1,274,811 input tokens, of which 1,193,216 were cached, and 6,873 output tokens: 88,468 net-new tokens under the benchmark definition. Its provider-reported total, including cached input, was 1,281,684 tokens. The root was then forked into L tasks; the one-time scout cost is never charged repeatedly. The scout prompt, visible events, final message, turn metadata, and warm-root record are archived.^[3, 4]

Prompt	SHA-256 prefix (full digest in campaign record)
Common statement	c3ece6dddf2b2c
L treatment appendix	579a68943cfb636
Task-neutral scout	f2517b44f0801c0
Proof-stage base	b21295790a3ff3f
Strict no-hole proof prompt	537ebad7810ec9a
Pilot 35 blind/direct/round-trip/adjudicator	campaign JSON, <code>inputs.audit_prompts</code>

Table 2: Prompt identity. The local files, campaign records, and logged effective prompts give the full text, not just these short display hashes.[15, 11, 12, 13, 14, 16]

2.3 Hardware, concurrency, and frozen software

Titan ran Linux/x86-64 on an Intel Core i9-13900K. Three disjoint lanes operated concurrently, each constrained to eight logical CPUs and 24 GiB of RAM, with swap disabled and a 512-process cap. Original schedule index alternated N/L order. CPU affinity, cgroup limits, before/after snapshots, and sampled per-turn peaks were logged. The maximum observed peak across condition/task turns was 5.85 cores and 2.35 GiB; sampling can miss brief spikes. These are lane peaks, not whole-host peak usage. Parallel execution reduces campaign makespan but does not divide or remove an individual contestant’s active time.[15, 16, 1]

Lean was `leanprover/lean4:v4.29.0-rc3`; Mathlib was commit `e8ea1afc32790ce1d4e1a4e45cc412ba9388716b`, and NumStability source commit was `45813a95dacf577461bae13f033af0dbc985a225`. The separate full-library build record reports 1,315.18 seconds after Mathlib dependency-cache preparation; it was not charged to task-active time. It records hardware, cache state, command, output digest, and resource counters. The exact source and runtime snapshots are hashed and archived as metadata; authentication secrets and large compiled object caches are not published. [21, 22, 23, 24]

3 Metric definitions and denominators

For a condition and task, formalization time is contestant-active time up to the frozen faithful statement, including ordinary library search and all repairs. Proof time begins only after the faithful statement is frozen and includes all proof turns and rejected submissions. Their sum is the primary task time. Compilation, semantic dossier construction, and the multi-agent faithfulness audit are recorded as excluded overhead; the headline total is *not* end-to-end campaign wall time. Net-new tokens are input minus cached input plus output, while full provider usage including cached input is retained separately. The warm-inclusive panel allocates the single L scout cost evenly over all 15 tasks solely for visual comparability; the aggregate and cumulative analysis charge it exactly once. N has no library scout. Code lines exclude blank and comment-only lines. All 15 pairs have two faithful statements and two complete proofs, so all paired proof comparisons have denominator 15.[9, 1, 3, 2]

Metric (15 tasks, summed)	N	L	L/N	L gain
Formalization seconds	2,044.7	2,398.5	1.173	-17.3%
Proof seconds	8,470.2	6,579.4	0.777	+22.3%
Total active seconds	10,514.9	8,977.9	0.854	+14.6%
Total + warm seconds	10,514.9	9,161.2	0.871	+12.9%
Formalization net-new tokens	542,141	1,180,836	2.178	-117.8%
Proof net-new tokens	1,260,741	1,062,126	0.842	+15.8%
Total net-new tokens	1,802,882	2,242,962	1.244	-24.4%
Total + warm net-new tokens	1,802,882	2,331,430	1.293	-29.3%
Statement code lines	514	552	1.074	-7.4%
Proof code lines	4,913	4,413	0.898	+10.2%

Table 3: Aggregate arithmetic; “gain” is $(N - L)/N$, so positive favors L. Warm-up is charged once, not once per task. Values come from the frozen result JSON and warm-root record.[1, 3, 2]

4 Time, token, and code outcomes

The per-task panels deliberately show the complete paired schedule, not only L wins. L was faster on 9/15 tasks and shorter in proof code on 8/15. It was slower in formalization by 17.3% in aggregate but faster in proof by 22.3%. Its formalization used 2.18 times N’s net-new tokens, while proof used 15.8% fewer. Thus the model did not simply transfer all cost into a faster proof: it spent substantially more text/attention reaching the faithful statement. Mean task time excluding warm-up was 701.0 seconds for N and 598.5 for L; including the scout amortized over 15 gives 610.7 for L. Mean total net-new tokens were 120.2 thousand for N and 149.5 thousand for L, rising to 155.4 thousand with the amortized scout.^[1, 3, 2]

The one-time cost is best read cumulatively. It is incurred once before the first task, not carried as an identical delay in each task. Figure 10 also reveals whether the observed aggregate time advantage survives this charge. It does. The token deficit does not vanish under amortization; cached prefix accounting is given separately, because treating cached tokens as zero resource use would conceal substantial prompt processing.^[3, 1, 2]

5 Declaration uptake and the overlap hypothesis

The declaration scan is based on *elaborated* constants in the final L statement and the kernel-accepted proof term, including proof-local helper bodies. It is not an import-line or text-match count. The final result reports substantive statement reach in 12/15 tasks and direct proof-term NumStability reach in 14/15. The exception P14-SHIFTED-SUM used no direct NumStability declaration in either final statement or proof, yet its time and code lines improved; that gain cannot be attributed to observed library reuse. Exact names are in `generated/declaration_scan.json` and in the proof dependency evidence.^[1, 9, 19, 2]

For visualization, “algorithm/format statement” is an explicit lexical indicator for the `fl_recursiveSum`, `fl_dotProduct`, `SumTree`, `FloatingPointFormat`, or least-squares backward-error families in substantive statement reach. “Error/bound proof” indicates a proof constant with an error, gamma, product-bound, suffix-product, or signed relative-error witness name. These are *transparent descriptive tags*, not a validated semantic overlap score. A greater number of reached declarations can reflect a longer proof or a larger helper dependency graph, not stronger mathematical usefulness.^[2, 1]

The raw proof-declaration count had weak rank association with total-time gain ($\rho = 0.147$), net-new-token gain ($\rho = 0.134$), and proof-line gain ($\rho = 0.382$), all at $n = 15$. Those are not evidence of a reliable monotone relationship. The limited attempt variation prevents a meaningful attempt correlation. Searches and tool activity are visible in archived contestant events, but the trial did not isolate a causal “query overhead” variable; therefore slower L tasks cannot automatically be blamed on search.^[2, 16]

Selection makes the pooled pattern especially fragile. Among the five previously favorable retained tasks, L proof code fell 37.6% and active time fell 34.0%. Among the ten newly screened tasks, L proof code was 6.7% *longer*, while active time was 2.8% lower. In the nine-task exploratory subgroup with both an algorithm/format statement indicator and an error/bound proof indicator, pooled proof lines fell 18.3%. But that subgroup contains all five favorable retained tasks. Restricting it to the four *new* tasks reverses the sign: L proof lines were 9.2% longer and active time 4.4% longer. These comparisons do not license the claim “sufficient overlap implies improvement.” They establish reuse and selected successes, plus a clear boundary to generalization.^[1, 5, 2]

The observed library can demonstrably help: CAST08-FIXED3 reused dot and recursive-summation interfaces and reduced proof code from 325 to 221 lines; CAST08-PROP3.1 used summation/error declarations and reduced 373 to 200; H20-8 used least-squares backward-error interfaces and reduced 243 to 167. Conversely, a high declaration count is not sufficient: RUMP12-THM3.4 has many finite-format proof constants and did improve, whereas the P14 shifted tasks often reached NumStability yet did not consistently shorten code. The P14-SHIFTED-SOFTMAX proof reached only norm declarations and used 493 versus 481 N proof lines. These case observations are traceable to the exact declaration list and paired source files, but explanation of mechanism needs manual trace review, not declaration counts alone.^[1, 20, 16]

6 Overhead, hardware, and limitations

The audit process was substantial rather than a shortcut: excluded auditing summed to 5,051 seconds and 5.11 million reported tokens for N task records, and 5,960 seconds and 5.59 million for L task records. These are sums of role calls, not additional contestant time or necessarily serial campaign wall time. Candidate

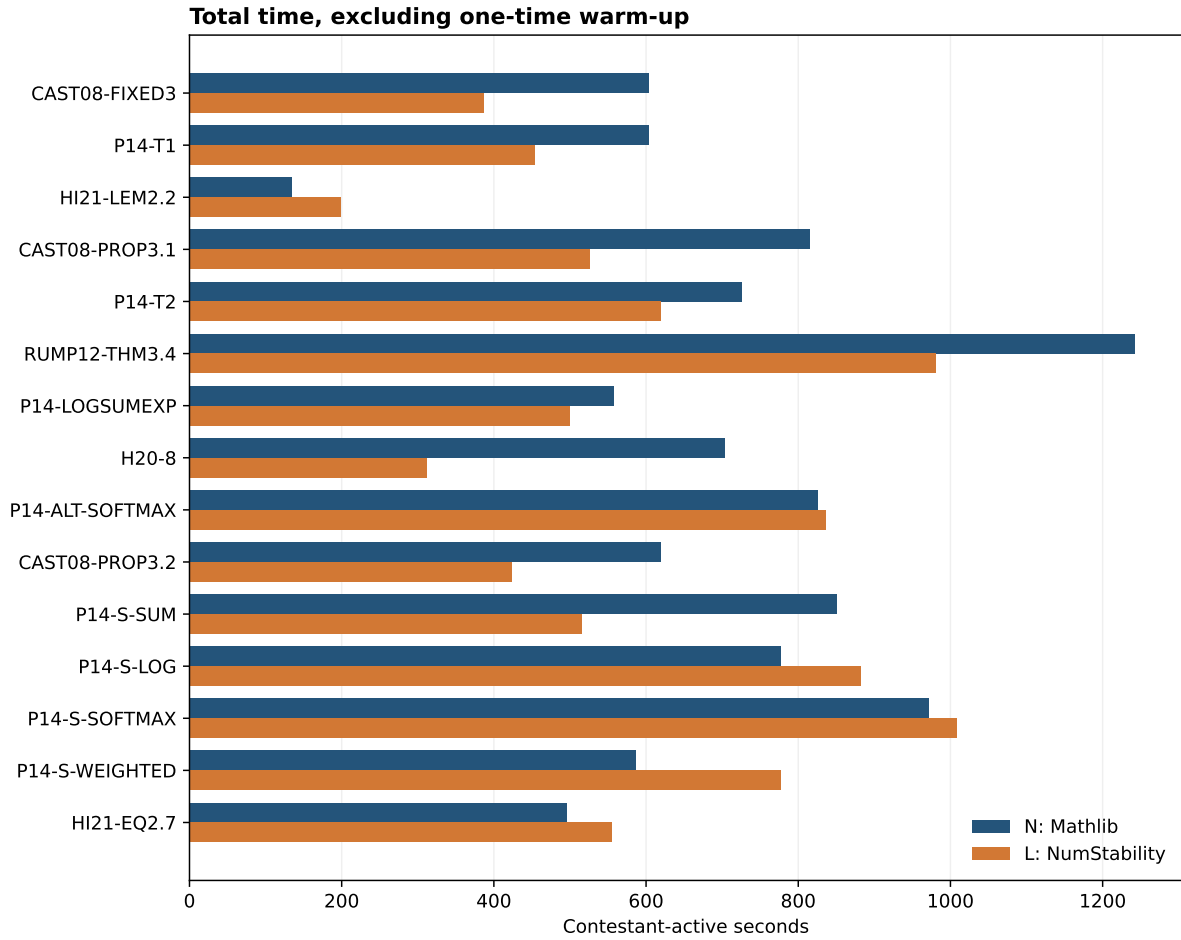


Figure 2: Per-task total contestant-active time, excluding the one-time L warm scout. Task-time library search remains included. All plotted task values come from the hash-checked final result and the reproducible plotting script, unless another source is named.^[1, 2]

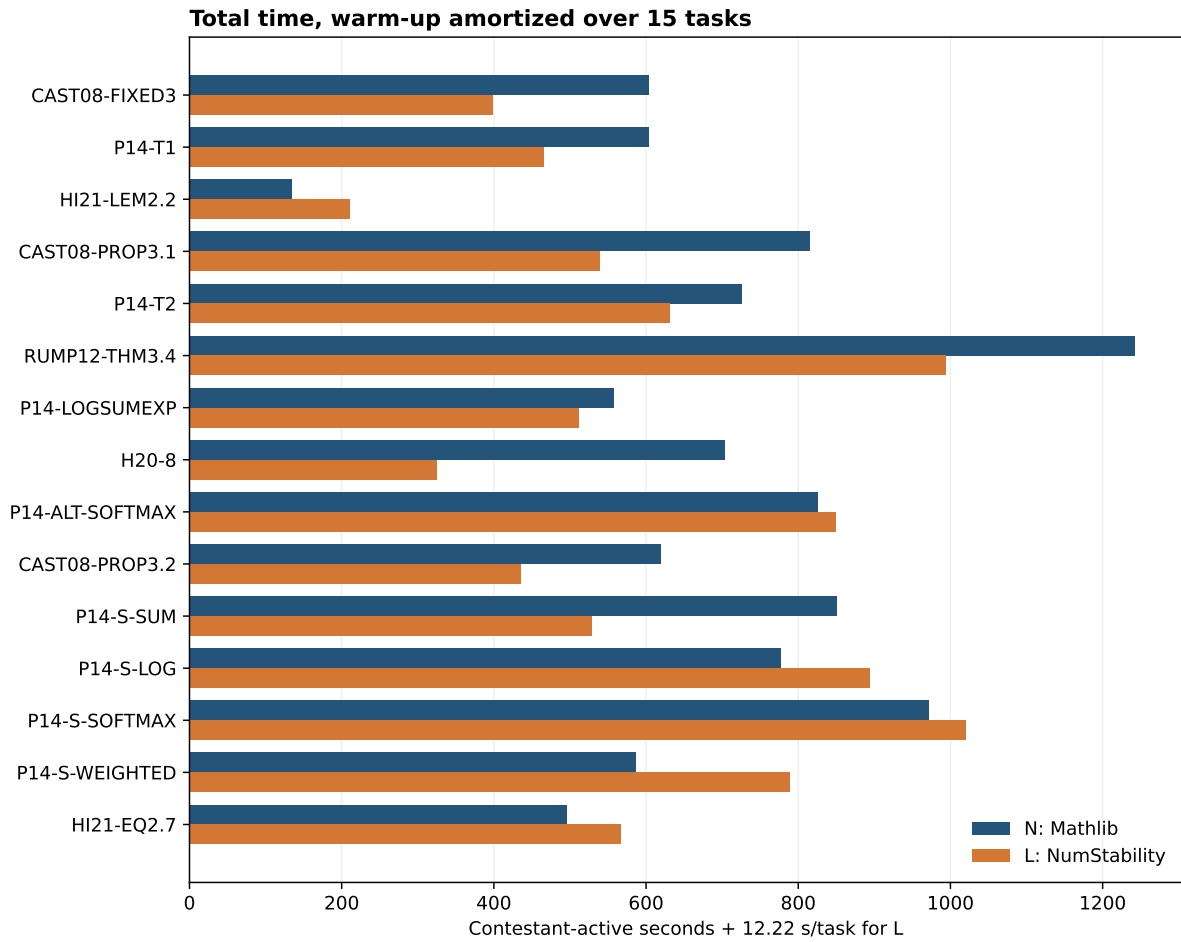


Figure 3: Per-task total contestant-active time with $183.313/15 = 12.221$ warm seconds added to every L bar. This is an allocation, not 15 repeat scouts. All plotted task values come from the hash-checked final result and the reproducible plotting script, unless another source is named.[1, 2]

compilation, semantic-dossier preparation, and proof validation have distinct logged fields. Full provider usage including cached input was 39.29 million tokens for N and 82.49 million for L across task formalization and proof stages; net-new tokens are the narrower primary comparison.^[1, 9]

Important limits are: (i) outcome-aware inclusion of five wins, (ii) eight correlated P14 tasks, (iii) one audit-schema incident and cross-campaign recovery, (iv) stronger-but-faithful statements in some pairs, (v) one run per task and no randomization, (vi) exact-reproduction dependence on a model service and authenticated Codex runtime, and (vii) reuse is observed, but a counterfactual “fine-tuned model without search cost” was not measured. The existing data support a modest thesis conclusion: the frozen library contains reusable numerical-stability components, and on several source- aligned tasks L produced shorter, faster proofs. They do *not* establish that the library will systematically outperform Mathlib-only formalization whenever declarations overlap, nor that expanding material coverage alone will remove search, representation, or interface costs.^[1, 5, 8, 2]

7 Reproducibility and source trail

The repository contains this report source, all plot-generation code and generated values, the frozen corpus and admission records, packets, prompts, controller and reporter sources, proof-dependency scanner, composite result JSON, the warm scout record/transcript, and compact archives of the 4+11 measured pairs. Each archive includes observed prompt files, formalizer events, Lean candidates, validators, audit outputs, pair reports, and hardware records; their checksums are in the companion README. The exact five source-PDF hashes are in `ADMISSION_15.json`. A new run must provide those PDFs, ChatGPT sign-in, and the pinned Lean/Mathlib/NumStability revisions. This public branch does not redistribute third-party book/article PDFs beyond those already in repository history, authentication material, compiled binary caches, or private Codex state. This permits a protocol rerun but not bit-for-bit reproduction of stochastic model conversations.^[25, 6, 16, 22, 23]

Rebuild all figures and numeric tables with the pinned Python requirements, then compile `report.tex` with `latexmk -pdf`. The script asserts that the 15 tasks match frozen schedule order and records SHA-256 of its two primary data inputs. Source-changing a measured task after seeing results would require a *new* pilot identity, not an edit to this result.^[2, 25, 5, 1]

A Task-level numeric ledger

Tables 4–5 are machine-generated from the final JSON, with rounded displays; plots use unrounded numbers. Formalization and proof times are contestant-active seconds. Token totals are net-new thousands. “Retained” records the five prior favorable tasks. The full CSV and JSON retain exact precision.^[1, 2]

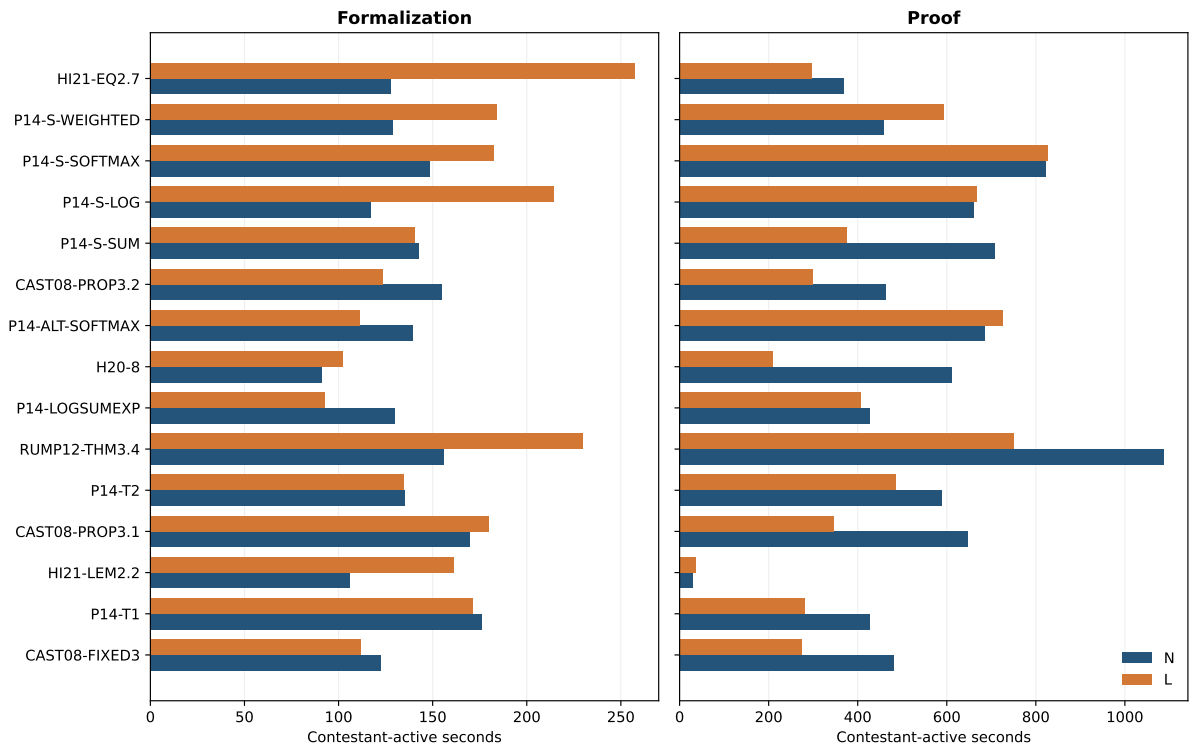


Figure 4: The formalization and proof components of each task’s active time, both inclusive of contestant work and excluding off-clock audit/validation. All plotted task values come from the hash-checked final result and the reproducible plotting script, unless another source is named.[\[1, 2\]](#)

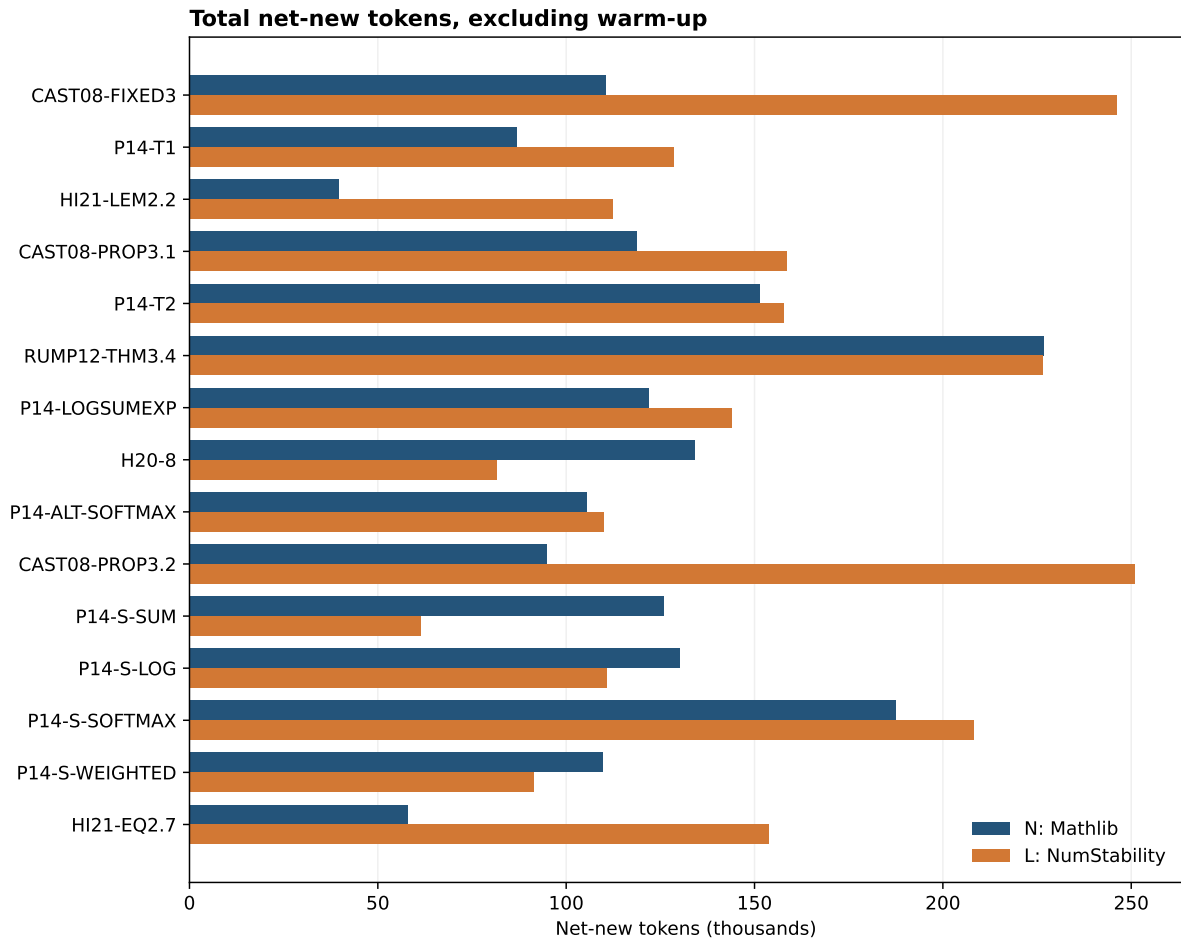


Figure 5: Per-task net-new formalizer tokens, excluding the one-time warm scout. Cached input is excluded by definition. All plotted task values come from the hash-checked final result and the reproducible plotting script, unless another source is named.[1, 2]

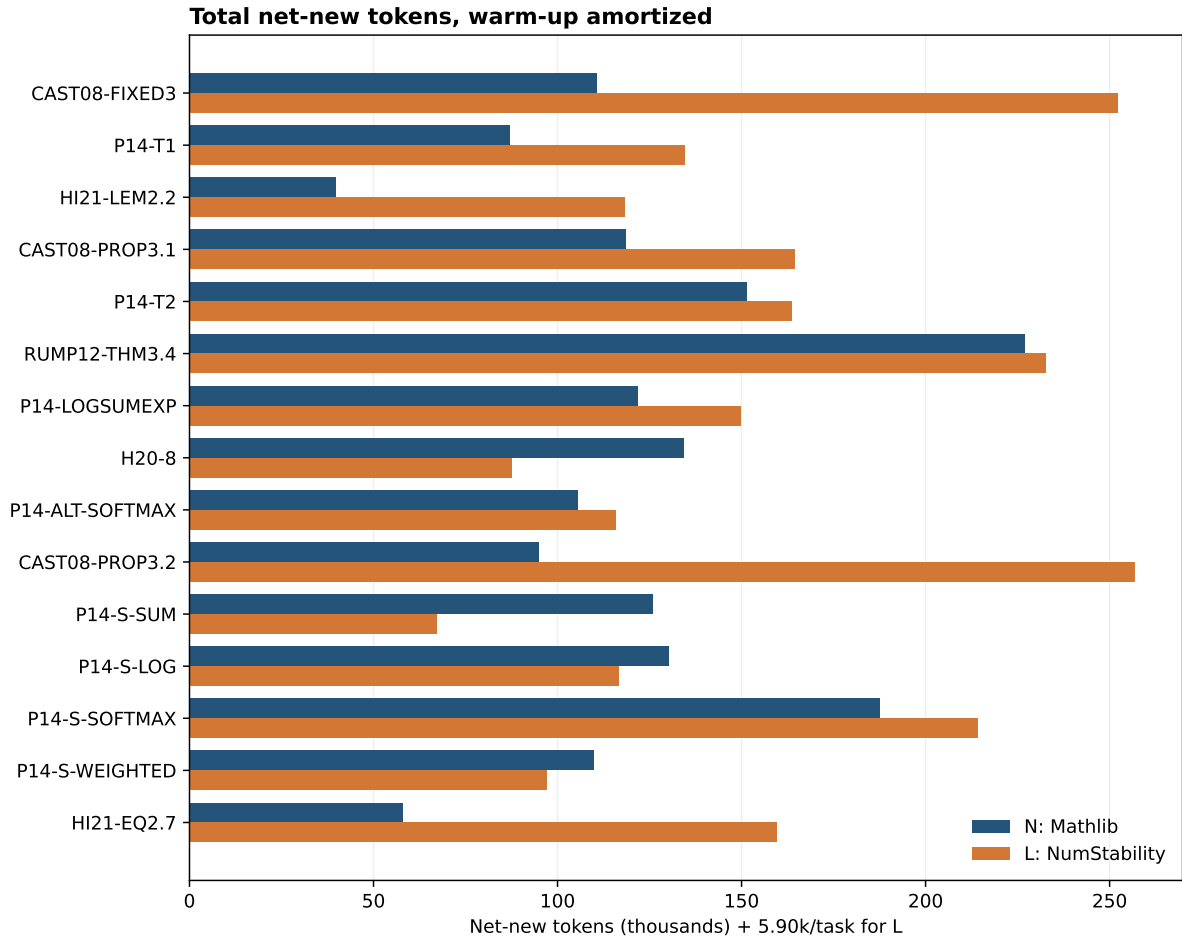


Figure 6: Per-task net-new tokens with $88,468/15 = 5,898$ one-time scout tokens allocated to every L task. All plotted task values come from the hash-checked final result and the reproducible plotting script, unless another source is named.^[1, 2]

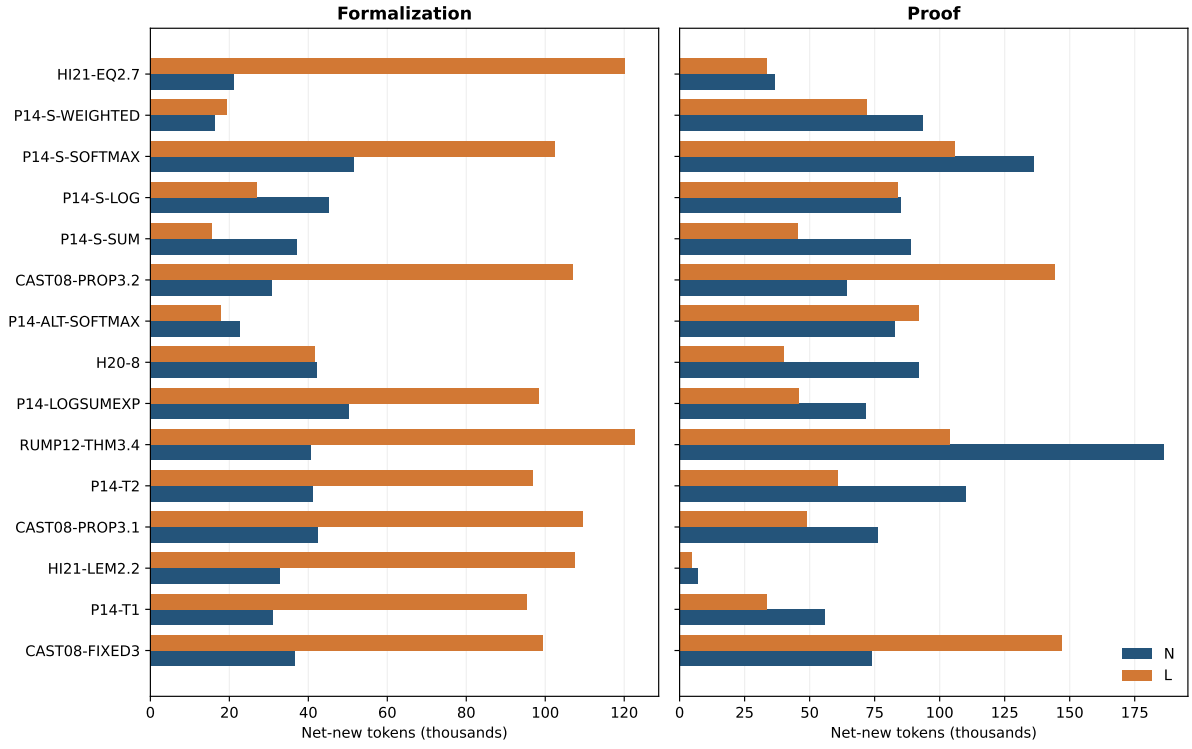


Figure 7: Formalization versus proof net-new tokens, showing the high L statement-stage token cost. All plotted task values come from the hash-checked final result and the reproducible plotting script, unless another source is named.[1, 2]

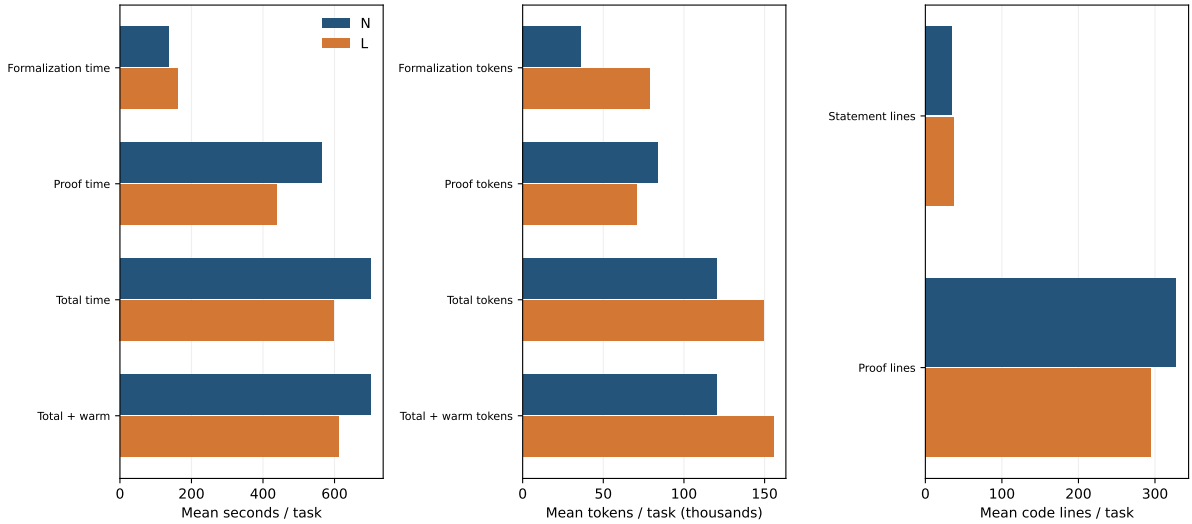


Figure 8: Arithmetic means per task for time, net-new tokens, and code lines. Warm-inclusive bars allocate the one-time scout over the full fifteen-task corpus. All plotted task values come from the hash-checked final result and the reproducible plotting script, unless another source is named.[1, 2]

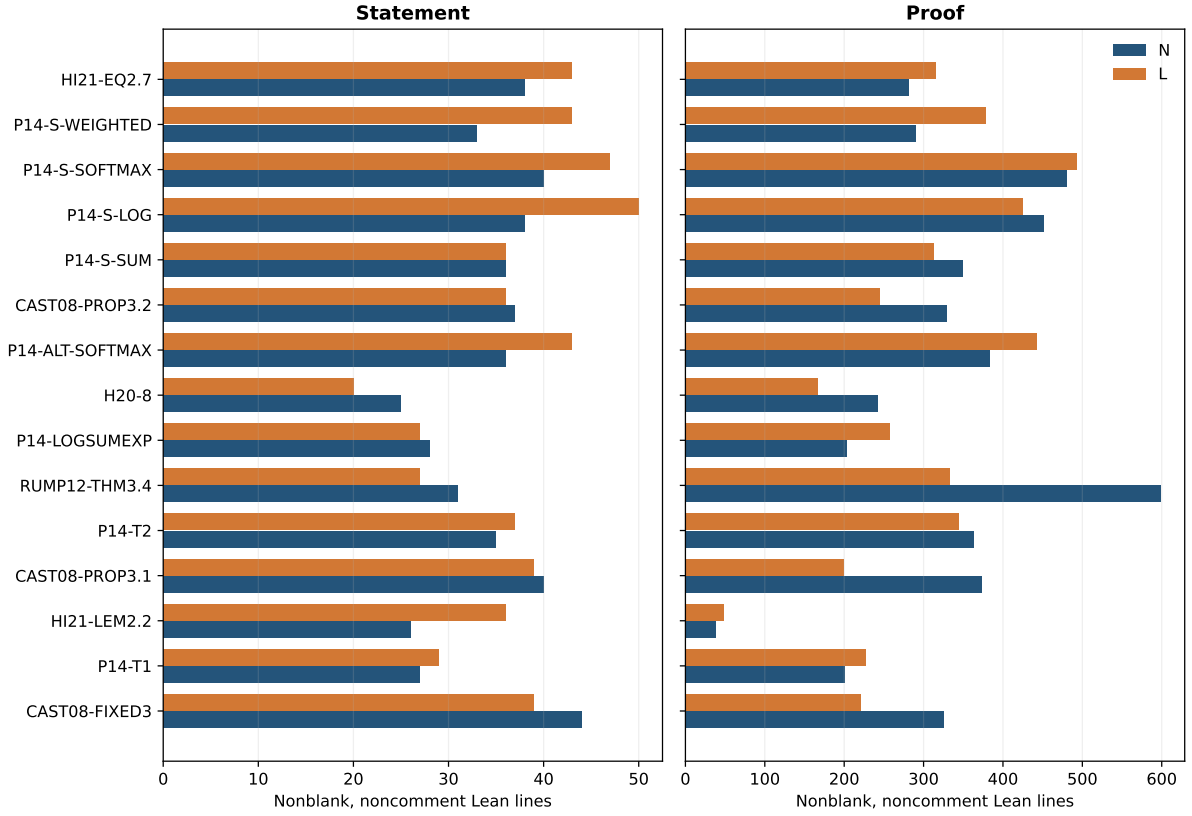


Figure 9: Final audited statement and complete proof code lines, nonblank and noncomment. Different accepted strengthening status can affect proof length. All plotted task values come from the hash-checked final result and the reproducible plotting script, unless another source is named.[1, 2]

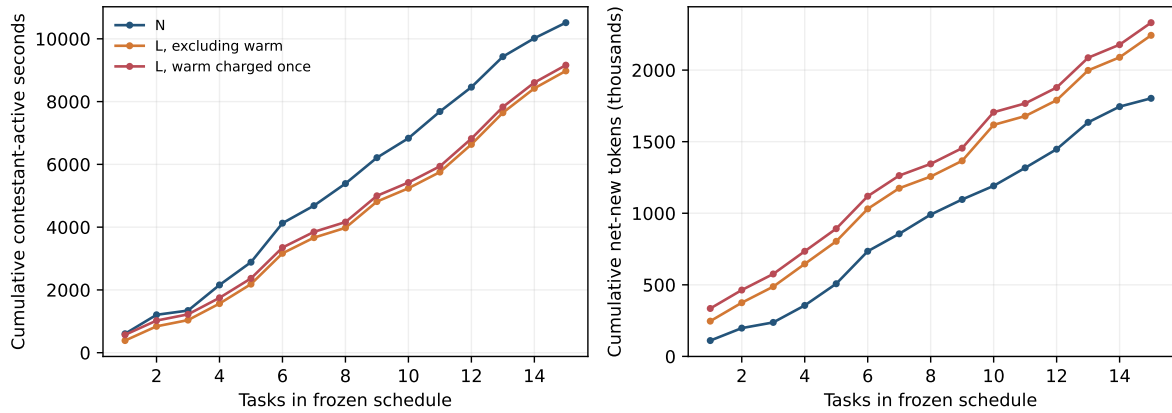


Figure 10: Cumulative active work in the frozen scheduled order. The third curve charges L's one-time scout once at the start. Not campaign makespan: lanes ran concurrently.[1, 3, 2]



Figure 11: Left: binary declaration-uptake indicators. Right: N-to-L gains, where blue/positive favors L and red/negative favors N. Both parts use the same task order. Uptake indicators are descriptive, not randomized treatments. All plotted task values come from the hash-checked final result and the reproducible plotting script, unless another source is named.[1, 2]

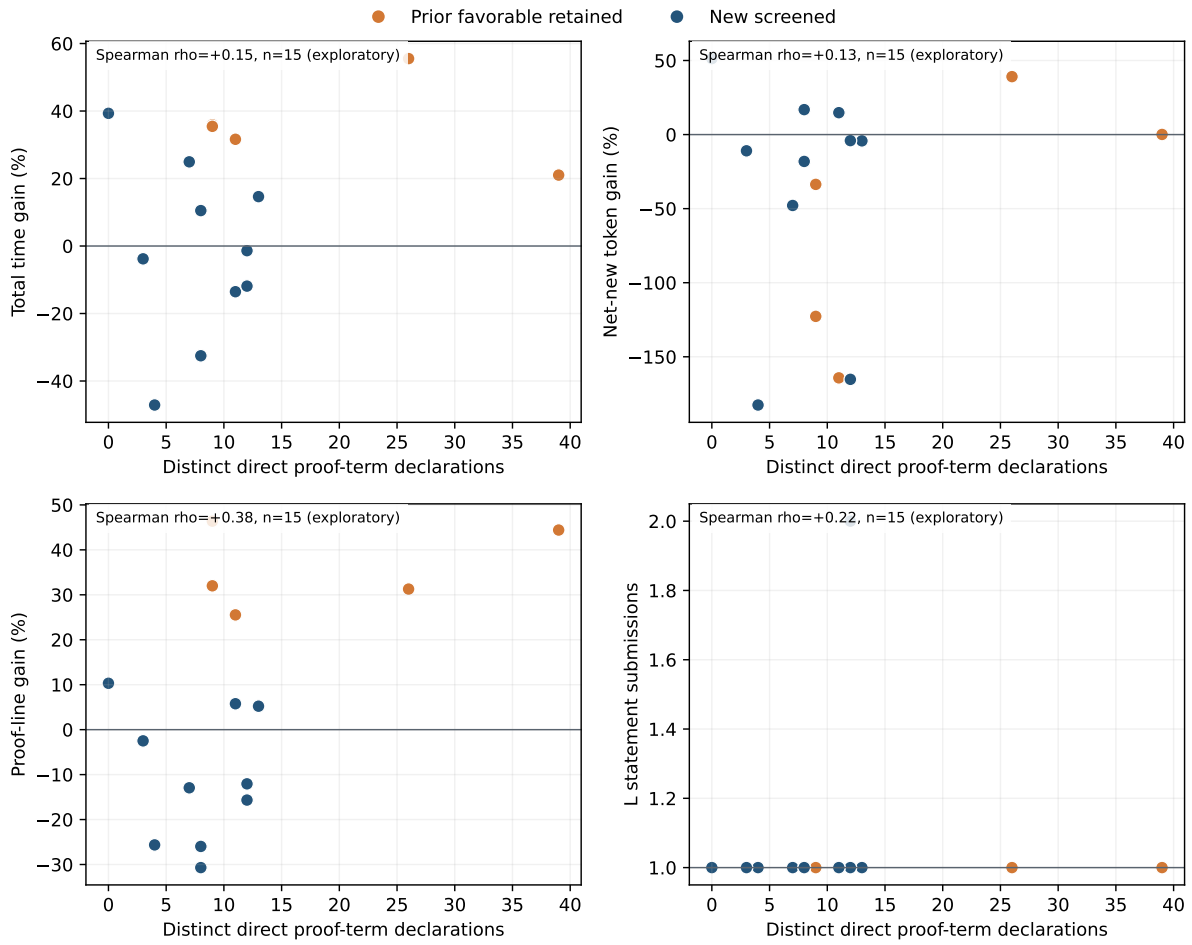


Figure 12: Distinct direct proof-term NumStability declarations versus paired gains and L statement submissions. Spearman coefficients are exploratory with $n = 15$; outcome-aware retained tasks are orange. Proof submissions were all one; statement submissions were all one except HI21-EQ2.7 L, which used two. All plotted task values come from the hash-checked final result and the reproducible plotting script, unless another source is named.[1, 2]

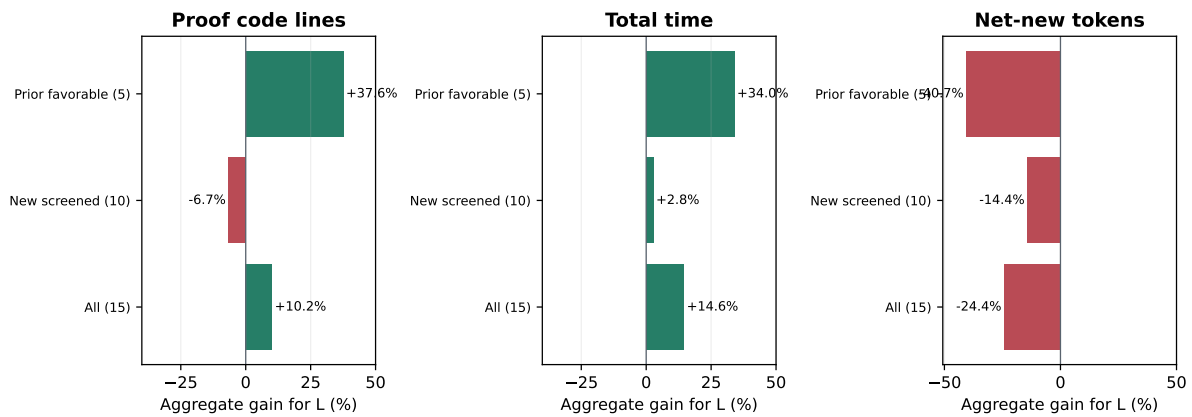


Figure 13: Aggregate N-to-L gains partitioned by the predeclared selection disclosure. Positive proof-code and time gains in the full sample are concentrated in five outcome-aware retained tasks. All plotted task values come from the hash-checked final result and the reproducible plotting script, unless another source is named.[1, 2]

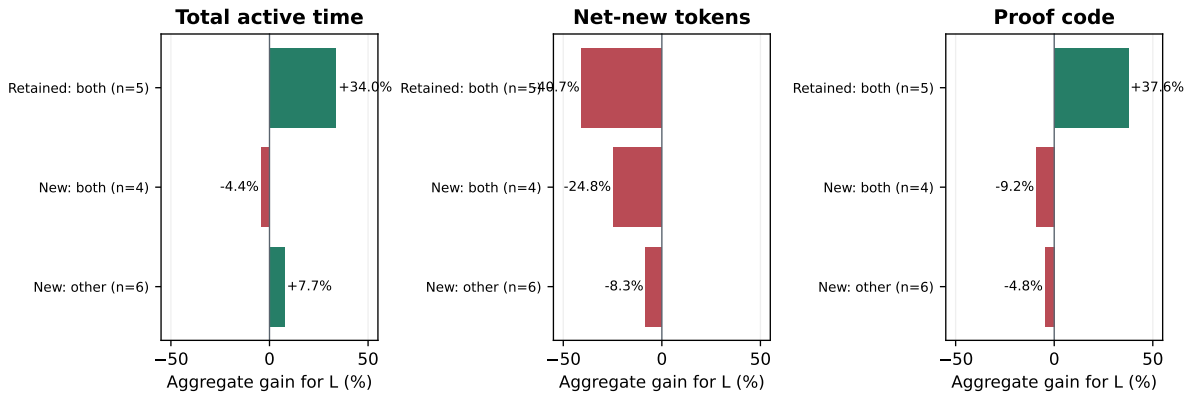


Figure 14: Sensitivity of the apparent overlap pattern to task selection. “Both” means the transparent algorithm/format-statement and error/bound-proof tags are both present. The new-only overlap subgroup has negative proof-code and time gains; this is an exploratory diagnostic, not a causal subgroup effect. All plotted task values come from the hash-checked final result and the reproducible plotting script, unless another source is named.[1, 2]

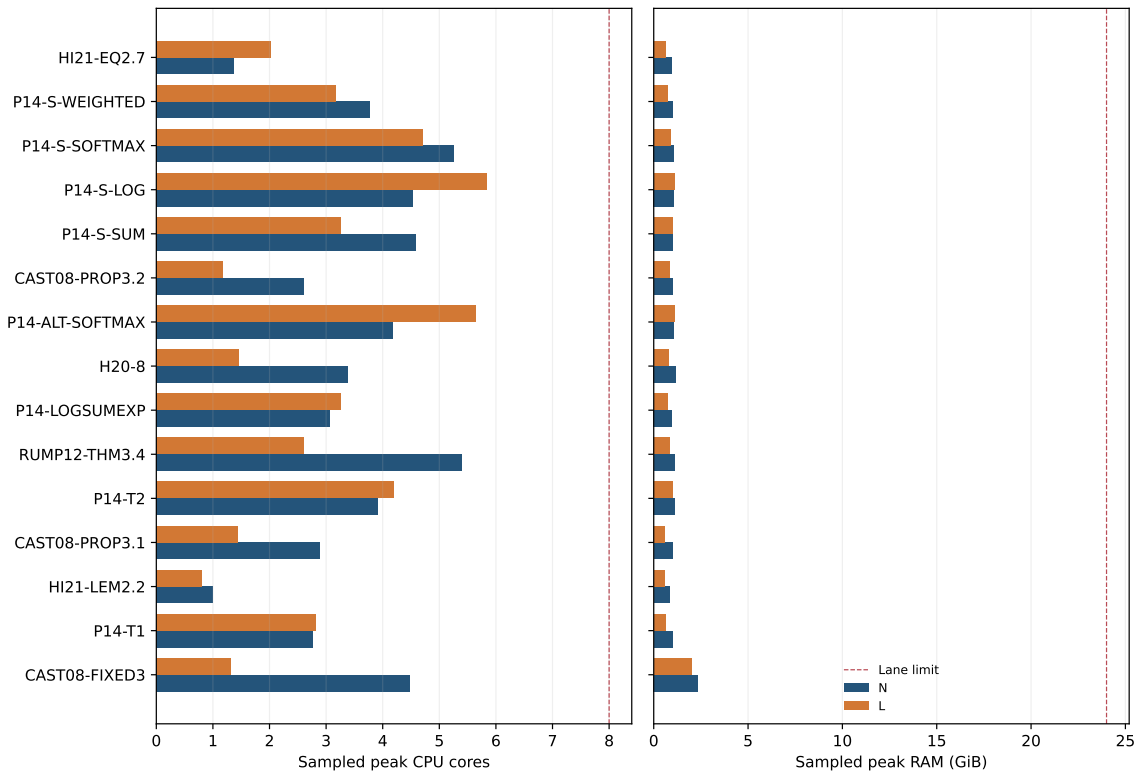


Figure 15: Sampled per-condition, per-task CPU and RAM peaks against lane caps. These cannot certify that unobserved compilation or audit spikes never occurred.[1, 15, 9]

Table 4: Task-level time and net-new token comparison, excluding warm-up.[1, 2]

Task	F-N	F-L	P-N	P-L	Total-N	Total-L	Tok-N(k)	Tok-L(k)
CAST08-FIXED3	123	112	481	275	603	387	111	246
P14-T1	176	171	428	282	604	453	87	129
HI21-LEM2.2	106	161	29	37	135	199	40	112
CAST08-PROP3.1	170	180	646	347	816	527	119	159
P14-T2	135	135	590	485	725	619	151	158
RUMP12-THM3.4	156	230	1087	752	1243	982	227	227
P14-LOGSUMEXP	130	93	428	407	558	499	122	144
H20-8	91	102	612	211	703	313	134	82
P14-ALT-SOFTMAX	140	112	686	725	826	837	106	110
CAST08-PROP3.2	155	124	464	300	619	423	95	251
P14-SHIFTED-SUM	142	141	708	376	851	516	126	61
P14-SHIFTED-LOG	117	215	660	668	777	882	130	111
P14-SHIFTED-SOFTMAX	148	183	823	826	972	1009	188	208
P14-SHIFTED-WEIGHTED	129	184	458	593	587	777	110	91
HI21-EQ2.7	128	257	369	298	496	555	58	154

Table 5: Task-level code and uptake. Stmt/Proof are nonblank code lines; SS is substantive statement declaration count, PT direct proof-term declaration count, and AF/AP are L’s formalization/proof submissions.[1, 2]

Task	Cluster	Retained	Stmt-N	Stmt-L	Proof-N	Proof-L	SS	PT	AF	AP
CAST08-FIXED3	CAST08	yes	44	39	325	221	2	9	1	1
P14-T1	P14	no	27	29	201	227	2	7	1	1
HI21-LEM2.2	HI21	no	26	36	39	49	4	4	1	1
CAST08-PROP3.1	CAST08	yes	40	39	373	200	1	9	1	1
P14-T2	P14	no	35	37	364	345	0	13	1	1
RUMP12-THM3.4	RUMP12	yes	31	27	599	333	7	39	1	1
P14-LOGSUMEXP	P14	no	28	27	204	257	2	8	1	1
H20-8	H20	yes	25	20	243	167	5	26	1	1
P14-ALT-SOFTMAX	P14	no	36	43	383	443	3	12	1	1
CAST08-PROP3.2	CAST08	yes	37	36	329	245	2	11	1	1
P14-SHIFTED-SUM	P14	no	36	36	349	313	0	0	1	1
P14-SHIFTED-LOG	P14	no	38	50	451	425	3	11	1	1
P14-SHIFTED-SOFTMAX	P14	no	40	47	481	493	1	3	1	1
P14-SHIFTED-WEIGHTED	P14	no	33	43	290	379	0	8	1	1
HI21-EQ2.7	HI21	no	38	43	282	316	5	12	2	1

B Declaration-usage index

This compact index groups exact names into readable families. The generated `declaration_scan.json` contains every direct elaborated name for every task, and the evidence archives contain the accepted Lean files. The families are labels only, not a validated semantic overlap score. No source-text-only mention is counted.^[20, 1, 16]

Table 6: Substantive L statement and direct proof-term declaration families; counts are exact, family labels descriptive.^[20, 1]

Task	Statement family	n	Proof family	n
CAST08-FIXED3	dot product, recursive sum	2	dot product, recursive sum, gamma bounds	9
P14-T1	recursive sum	2	recursive sum	7
HI21-LEM2.2	sum tree	4	sum tree	4
CAST08-PROP3.1	recursive sum	1	recursive sum, gamma bounds	9
P14-T2	none	0	suffix error, gamma bounds, product error	13
RUMP12-THM3.4	finite FP format	7	finite FP format	39
P14-LOGSUMEXP	recursive sum	2	recursive sum, gamma bounds	8
H20-8	LS backward error	5	LS backward error	26
P14-ALT-SOFTMAX	recursive sum	3	recursive sum, gamma bounds	12
CAST08-PROP3.2	recursive sum	2	recursive sum, gamma bounds	11
P14-SHIFTED-SUM	none	0	none	0
P14-SHIFTED-LOG	recursive sum	3	recursive sum, gamma bounds	11
P14-SHIFTED-SOFTMAX	infinity norm	1	infinity norm	3
P14-SHIFTED-WEIGHTED	none	0	suffix error, gamma bounds	8
HI21-EQ2.7	sum tree	5	sum tree, gamma bounds, product error	12

C Source registry and immutable identities

Every numeric table or chart is derived from the final machine-readable result. Every methodological claim is backed by frozen protocol, campaign, prompt, or hardware metadata. Short paths in the table are relative to `paper_benchmark/`; full SHA-256 values are in the companion README and the cited artifact records.^[25]

Source	Path / identity
Final result	<code>paper_benchmark/pilot35/RESULTS_15.json</code> ; SHA-256 begins 3faeb35af66f5c3c.
Warm root	<code>thesis_report/evidence/warm-root.json</code> ; SHA-256 begins 3401426d25ee278c.
Frozen task set	<code>formalization_benchmark/design33/CORPUS_15.json</code> and <code>ADMISSION_15.json</code> , both under <code>paper_benchmark/</code> .
Pair evidence	Two compressed Pilot 34/35 archives under <code>paper_benchmark/thesis_report/evidence/</code> ; exact names and hashes in README.
Library build	<code>thesis_report/evidence/library-build-record.json</code> ; SHA-256 begins 6f4581380982e51f.
Runtime snapshot	<code>thesis_report/evidence/runtime-snapshot.json</code> ; SHA-256 begins 9fac78511309a591.
Controller commits	Pilot 34 cfec400f94f4b957; Pilot 35 789c7b5b7513d178; full commits in archived campaign JSON.

References

- [1] Pilot 35 composite result, `paper_benchmark/pilot35/RESULTS_15.json`, SHA-256 `3faeb35af66f5c3c9d16e0800a4c88bd5cef09e1a35f0bc10e6c387f40d9c050`.
- [2] This branch, `paper_benchmark/thesis_report/make_figures.py`, generated numeric CSV/JSON and PDFs in `paper_benchmark/thesis_report/generated` and figures.
- [3] Frozen warm-root record, `paper_benchmark/thesis_report/evidence/warm-root.json`, SHA-256 `3401426d25ee278cbf33e48da80f665d5bcc4ad8bcab721a4fa69cae0095ee39`.
- [4] Scout prompt and observable transcript, `paper_benchmark/thesis_report/evidence/warm-scout/`.
- [5] Frozen corpus and selection disclosure, `paper_benchmark/formalization_benchmark/design33/CORPUS_15.json`.
- [6] Admission and source-PDF hashes, `paper_benchmark/formalization_benchmark/design33/ADMISSION_15.json`.
- [7] Fifteen source packets with bibliography, selected result and paper location, `paper_benchmark/formalization_benchmark/design33/packets/`.
- [8] Prospective screening ledger, `paper_benchmark/formalization_benchmark/design33/SCREENING.md`.
- [9] Prospective metrics contract, `paper_benchmark/formalization_benchmark/design33/METRICS.md`.
- [10] Proof-required benchmark protocol, `paper_benchmark/formalization_benchmark/design27/PROTOCOL.md`.
- [11] Frozen common formalization prompt, `paper_benchmark/formalization_benchmark/design27/prompts/common.md`.
- [12] Frozen L appendix, `paper_benchmark/formalization_benchmark/design27/prompts/library_appendix.md`.
- [13] Strict proof prompt, `paper_benchmark/prospective_proof_no_holes/PROOF_PROMPT.md`.
- [14] Audit role prompts, `paper_benchmark/formalization_benchmark/audit/prompts/`; campaign-specific hashes in archived `campaign.json`.
- [15] Archived Pilot 34 and 35 `campaign.json` and hardware-before records in the two evidence bundles. Pilot 35 input identity `6caea657f3c4a8139fbf90f6737e2c40d9311ebe7d6592007b4749af73c56f1d`.
- [16] The two measured-pair evidence archives under `paper_benchmark/thesis_report/evidence/`, with SHA-256 and member counts in the companion README.
- [17] Recovery manifest and incident protocol, `paper_benchmark/pilot35/RECOVERY_11.json` and `RECOVERY_PROTOCOL.md`.
- [18] Frozen composite reporter, `paper_benchmark/pilot35/composite_report.py`.
- [19] Elaborated proof dependency scanner, `paper_benchmark/formalization_benchmark/tools/design33_proof_dependencies.lean`; scanner digest in final result.
- [20] Generated declaration ledger, `paper_benchmark/thesis_report/generated/declaration_scan.json`.
- [21] NumStability full build record, `paper_benchmark/thesis_report/evidence/library-build-record.json`.
- [22] Library snapshot, `paper_benchmark/thesis_report/evidence/library-snapshot.json`.
- [23] Runtime snapshot, `paper_benchmark/thesis_report/evidence/runtime-snapshot.json`.
- [24] Titan deployment hash and Codex version, archived campaign input `deployment_sha256` and build/runtime records.
- [25] Rebuild and evidence instructions, `paper_benchmark/thesis_report/README.md`.
- [26] A. M. Castaldo, R. C. Whaley, A. T. Chronopoulos, “Reducing Floating Point Error in Dot Product Using the Superblock Family of Algorithms,” *SIAM J. Sci. Comput.* 31(2), 1156–1174 (2008). Author copy: <https://www.cs.utsa.edu/~atc/pub/J42.pdf>.
- [27] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM (2002), Problem 20.8. <https://epubs.siam.org/doi/book/10.1137/1.9780898718027>.
- [28] E. Hallman and I. C. F. Ipsen, “Deterministic and Probabilistic Error Bounds for Floating Point Summation Algorithms” (2021). <https://arxiv.org/abs/2107.01604>.
- [29] P. Blanchard, D. J. Higham, N. J. Higham, “Accurately Computing the Log-Sum-Exp and Softmax Functions,” *IMA J. Numer. Anal.* 41(4), 2311–2330 (2021). <https://doi.org/10.1093/imanum/draa038>.
- [30] S. M. Rump, “Error Estimation of Floating-Point Summation and Dot Product,” *BIT Numer. Math.* 52, 201–220 (2012). Author copy: <https://www.tuhh.de/ti3/paper/rump/altRu11.pdf>.